

AI AGENT · 실행 환경 설계 · 데이터 파이프라인

실패의 원인을 깊이 파고들어 해결하는 것을 좋아하는 개발자

각 프로젝트에서 마주한 문제, 해결을 위한 시도와 결과, 그에 따른 판단과 구현 과정을 담았습니다.

김재형

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com · martinel2.github.io

LLM 워크스페이스의 AI 기능 전체를 맡아 RAG와 에이전트, 품질 평가 체계를 설계하고, AI 코딩 도구로 구현·검증했습니다.

정확도와 비용, 응답 시간의 트레이드오프를 평가로 확인한 뒤 서비스에 맞는 구현을 선택합니다.

RAG 근거 선택: 근거 충족률 66.25% → 91.25%, 잘못된 근거 20 → 0건

골든셋을 만들고 동일 후보·블라인드·두 모델 교차채점으로 조건을 통제된 오프라인 비교 평가를 거쳐 근거 선택 단계를 교체했습니다. 후보 수와 병렬 처리를 조정해 선택 시간 중앙값 9.090초 → 2.251초.

에이전트 실행 환경: 사용자 정의 Skill과 승인 기반 실행, 편집 품질 82.5% → 91.2%(114건 평가)

권한과 프롬프트 주입 검사는 코드가, 작업 지침과 수정안 생성은 AI가, 게시와 실제 반영은 사용자가 맡도록 책임을 나눴습니다. 사용자가 승인하기 전에는 문서를 바꾸지 않습니다.

데이터 파이프라인: PDF 변환 통과율 45.17% → 89.89%, 약품 변환 비용 예상 \$200 대비 \$11.18

본문·표·수식의 처리 경로를 나눠 문서를 검색과 답변에 쓸 수 있게 변환하고, 중복 제거와 Batch 변환으로 약 4만 4천 건을 예산 안에서 처리했습니다.

목차

시간이 없으시면 01·02·04를 먼저 보시면 됩니다. 표지의 세 가지가 각각 품질 평가, 에이전트 실행 환경, 데이터 파이프라인입니다.

FRUITION

01	RAG 근거 선택 - 충족률 66.25% → 91.25% 개선	p.6
02	에이전트 실행 환경 - 지침 작성·검증·승인 설계	p.12
03	작업 취소 - 저장한 변경의 역순 복구	p.17
04	PDF 변환 - 통과율 45.17% → 89.89% 개선	p.21
05	문서 병렬 분석 - 처리시간 73.8% 단축	p.25
06	에이전트 루프 - PR 16개 5일 병합	p.27

PILLTIP

07	의약품 정제 - 예상 \$200 대비 \$11.18로 변환	p.34
08	복약 챗봇 - 건강 프로필의 외부 전송 제외	p.39

MORE

09	코드 밖에서도 이어지는 경험	p.42
	Rhwp · HWPX 저장 오류 수정 APPTIVE · 백엔드 멘토링 백준 945일 연속 문제 해결	
10	프로젝트 회고	p.43

Fruition

업로드한 문서를 LLM이 Wiki로 정리하고, RAG 기반으로 원문 근거를 찾아 답변하거나 자연어로 문서를 편집하는 AI 워크스페이스입니다. 현재 AWS EKS에 배포해 QA를 진행하고 있습니다.

시작 계기

팀원이 제기한 문서 관리의 불편함을 디자인 싱킹으로 구체화하며 시작했습니다. 문서를 저장만 해 두고 정작 필요할 때 찾지 못하는 불편을 해소하는 것이 목표였습니다.

GitHub · AI 분리 저장소 ↗

팀 구성과 내 역할

MY ROLE

AI

PDF 변환, Wiki 생성, RAG 검색·근거 답변, 문서 편집 Agent 등 AI 기능 전체

01 AI 기능 전체 리드

PDF문서 변환, Wiki 생성·정리, 검색·근거 답변, 문서 편집 Agent 등 AI 기능 전반의 설계·평가를 리드하고, 구현은 AI 코딩 도구로 진행했습니다.

02 MSA 설계 기여

제가 초기 아키텍처를 설계·제안한 뒤, 팀원·멘토와 서비스 경계, 데이터 소유권, 통신 방식을 논의해 최종 구조를 함께 완성했습니다.

백엔드 1명

Spring 기반 백엔드 전체

프론트엔드·DevOps 1명

프론트엔드 전체, MSA 기반 AWS 배포

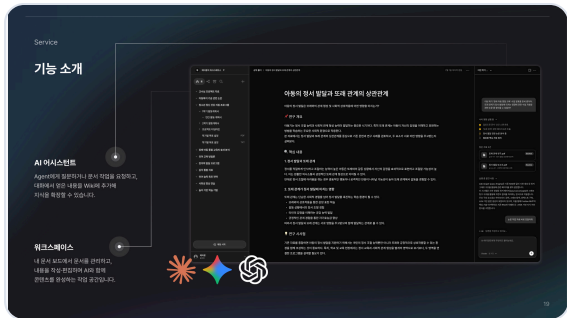
디자인

외주 디자이너

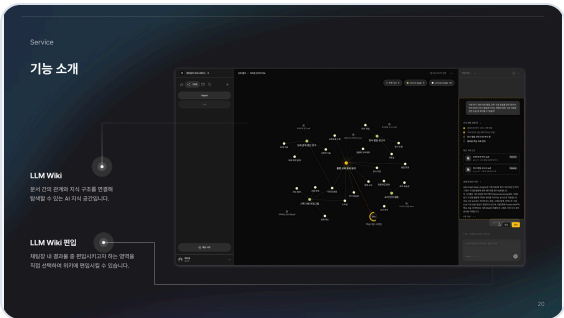
멘토 3명

Gen AI 품질·평가 / 백엔드 전반 / 마일스톤·MSA 설계·문서화

담당 기능과 서비스 화면



문서 편집·AI 어시스턴트
요청을 해석하고 수정안을 생성·검증하는 AI 기능을 리드했습니다. 편집기와 어시스턴트 UI는 팀 구현입니다.



Wiki 생성·검색
문서를 지식으로 정리하고 연결해 질문에 맞는 근거를 찾는 AI 기능을 리드했습니다. 그래프와 검색 UI는 팀 구현입니다.

Architecture

시스템 아키텍처

브라우저는 Spring Boot API를 통해 데이터와 AI Service을 사용합니다.

기술	구현
Next.js	문서 · Wiki · Chat · Agent UI
Spring Boot	인증 · 권한 · 워크스페이스 · 상태
FastAPI	Ingest · Query · Lint · 문서 편집 Agent
PostgreSQL	관계 · 버전 · 작업 상태
MinIO	원본 · Markdown · artifact

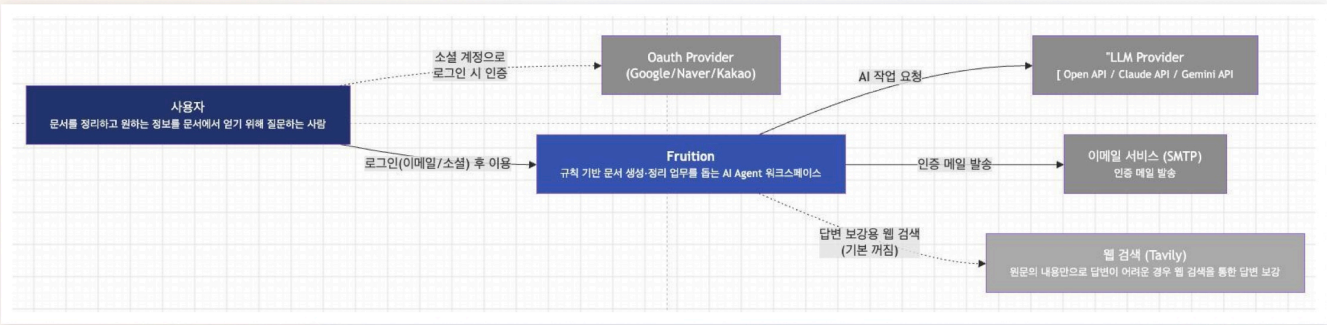
14

서비스 역할 구분

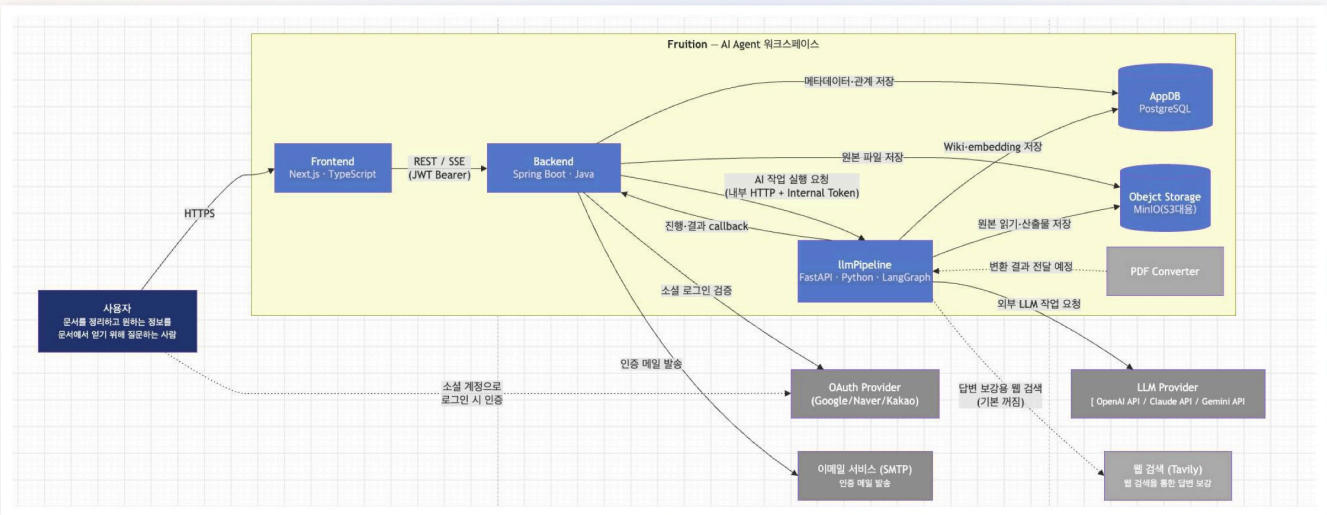
AI 기능 전체를 리드했고 프론트엔드·백엔드는 팀원이 담당했습니다. MSA는 제 초기 설계를 바탕으로 팀원·멘토와 논의해 완성했습니다.

시스템 아키텍처

| C1 전체 시스템 아키텍처 간략화



| C2 Fruition 시스템 일부 구체화



위는 전체 시스템, 아래는 Fruition 내부 구성입니다. 제 담당은 llmPipeline(FastAPI·Python·LangGraph)을 중심으로 한 AI 기능이며, 프론트엔드·백엔드와 AWS 배포는 팀원이 담당했습니다.

RAG 근거 선택 - 충족률 66.25% → 91.25% 개선과 처리시간 75% 단축

정해진 선택지 중에서만 답을 고르는 판단 전용 모델 Jev를 요청 라우팅에 먼저 적용해, 같은 선택지를 준 기존 JSON 방식 77/98건·GPT-5-nano 63/98건 대비 81/98건의 정확도와 더 짧은 판단 시간을 확인했습니다. 이를 계기로 RAG의 답변 근거 선택에도 적용했고, 근거 충족률을 66.25%에서 91.25%로 높였습니다. 늘어난 처리 시간은 후보 수와 병렬 처리 조정으로 줄였습니다.

2026.09

Jev 1.13.0

BGE-M3

BM25

RAG

품질 평가

배경

사용자 요청에 맞는 작업과 답변 근거를 안정적으로 선택하기 위해, 라우팅부터 RAG까지 Jev의 적용 범위를 넓혀가며 검증했습니다. 검색은 의미·키워드 검색과 Wiki 그래프 탐색으로 원문 근거를 모으는 고정 파이프라인으로 두고, 답변과 근거·관련 페이지·탐색 경로를 분리해 보여주도록 설계했습니다. 검색 전략을 스스로 계획하는 Query Agent는 보류했습니다. 에이전트를 늘리는 기준은 한 질문이 여러 검색 계획과 중간 결과에 따른 재계획을 요구할 때로 정했고, 그 전까지는 고정 파이프라인이 더 적은 호출로 같은 근거를 모으고 실패 지점도 추적하기 쉽다고 판단했습니다.

근거 충족률(Golden set을 통한 LLM 평가)

66.25% → 91.25%

Jev 근거 선택 시간

중양값
9.090초 → 2.251초

답 없는 질문 20건에 대한 근거 반환

20건 → 0건

실험과 개선 과정

1 요청 분류 성능 비교와 RAG 적용 결정

문제 상황

사용자 질문이 문서 편집으로 분류되는 등, 요청에 맞지 않는 작업으로 이어지는 문제가 있었습니다. 작업 종류와 부가 조건을 더 정확하고 빠르게 고를 방법이 필요했습니다.

시도와 결과

9/14일 출시된 선택형 판단 모델 Jev를 사용자 요청을 질문, 문서 편집, 문서 생성, 폴더 정리 등으로 나누는 라우팅에 적용했습니다.

모델이 판단한 98개 요청에서 작업 종류와 부가 조건을 모두 맞힌 수는 기존 JSON 방식 77개, 같은 선택지를 받은 GPT-5-nano 63개, Jev 81개였습니다. 판단시간 중앙값 또한 기존 8.967초에서 Jev 0.658초로 개선된 결과를 보였습니다.

같은 100문항으로 API 비용도 확인했습니다. Jev 약 \$0.0408, 같은 선택지를 준 GPT 방식 약 \$0.1027이었습니다.

판단

Jev는 정확도와 속도, 비용 모두 가능성을 보였습니다. 이 결과를 바탕으로, 기존 검색에서 해결하지 못한 답변 근거 선택에도 Jev를 적용해보기로 했습니다.

다른 방법을 두고 Jev를 고른 이유는 두 가지였습니다. GPT에게 근거를 고르게 하면, 같은 선택지를 준 비교에서도 비용이 약 2.5배였고 판단 시간도 길어 질문마다 수백 개 후보를 검사하는 단계에는 부담이 너무 컸습니다. 검색 점수를 조정해 순위를 다시 매기는 방향은, 하이브리드 가중치와 RRF를 여러 값으로 실험해 봐도 점수가 비슷한 수준에 머물렀습니다. 임베딩이 한국어 표현의 차이를 충분히 구분하지 못하는 경우가 있어, 분야마다 성격이 다른 질문에 하나의 가중치를 맞추는 것으로는 개선 폭이 작다고 봤습니다. 그래서 순위를 더 다듬기보다, 모아 온 후보 중에서 필요한 근거를 직접 판단하게 하는 쪽을 택했습니다.

실험과 개선 과정 (계속)

2 전체 근거 검사로 평가 질문 90건의 근거 총족

문제 상황

질문 원문을 검색에 사용하고 의미 점수의 비중을 높여, 질문 110개에서 정답 자료가 첫 번째에 나오는 경우를 50개에서 57개로 향상시켰습니다. 하지만 이는 일부 편향된 실험이었으며, 새로운 질문에서도 필요한 근거를 안정적으로 고르는 문제는 해결하지 못했습니다.

시도와 결과

라우팅에서 효과를 확인한 Jev가 검색 단계에서 놓친 근거도 찾을 수 있는지 확인했습니다. 테스트용 위키에 저장된 근거 2,020개를 검색으로 거르지 않고 모두 검사한 뒤, 기존 검색과 동일하게 최대 8개를 반환했습니다.

의역을 인정한 의미 평가에서 답이 있는 90개 질문의 근거 총족률은 기존 24/90건(26.7%)에서 Jev 90/90건(100%)으로 높아졌습니다. 답이 없는 질문 10개에는 모두 근거를 반환하지 않았습니다. 처리시간 중앙값은 기존 1.511초, Jev 5.338초였습니다. Jev는 80개씩 나눈 26개 묶음을 최대 4개씩 병렬 처리했습니다.

전수 검사는 100질문에 2,600회를 호출했고, API가 보고한 사용량은 입력 약 1억 2,448만 토큰, 출력 약 647만 토큰이었습니다. 실제 청구액은 확인하지 않았습니다.

차이가 큰 이유를 확인하려고 기존 검색의 탈락 단계도 추적했습니다. 정답 문장을 가진 페이지는 90문항 모두 초기 후보에 있었지만, 76문항은 검사 대상을 앞 8개 페이지로 제한하는 단계에서, 8문항은 페이지 내부 최고 점수의 85% 기준에서, 나머지 2문항은 최종 근거 8개 제한과 관련 페이지 탐색에서 빠졌습니다.

판단

기존 검색은 선택 전에 정답을 이미 버리고 있었습니다. 따라서 이 차이를 동일 후보에서의 선택 능력 차이로만 해석할 수는 없었습니다. 전체 근거를 보면 필요한 정보를 찾을 수 있었지만, 질문마다 모든 근거를 검사하는 방식은 문서가 늘수록 호출량도 늘어납니다. 이 테스트 위키의 90문항 결과만으로 적용 범위를 정하지 않고, 검색으로 후보를 먼저 줄인 뒤 Jev가 답변 근거를 선택하도록 바꾸기로 했습니다.

실험과 개선 과정 (계속)

3 동일 500개 후보 비교에서 근거 총족률 87.50% 달성

문제 상황

전체 근거 실험은 질문마다 26묶음을 호출해야 했습니다. 또한 특정 기술 문서에 질문이 몰려 있었고, 질문에 문서 제목이 포함돼 있었습니다. 기존 검색은 일부 페이지만 살피는 반면 Jev는 전체 근거를 봤으므로, 정확도 차이가 모델의 판단력 때문인지 접근 범위 때문인지 구분하기 어려웠습니다. 후보를 줄이는 것과 함께 비교 조건도 맞춰야 했습니다.

시도와 결과

논문 9편으로 질문 100개를 새로 만들고, 기존 평가 질문은 재사용하지 않았습니다. 한 논문에서 답을 찾는 질문 63개, 두 논문을 연결해야 하는 질문 17개, 답이 없는 질문 20개로 구성하고 질문·정답·필수 사실을 실행 전에 고정했습니다.

중복 근거를 합친 뒤 BGE-M3와 BM25로 최대 500개 후보를 한 번만 추려 두 방식에 같은 본문과 정보를 제공했습니다. 기존 선택기도 후보에 속한 모든 페이지를 볼 수 있게 했고, 양쪽 모두 최대 8개 근거의 본문을 자르지 않고 반환하도록 맞췄습니다. 정답은 검색과 Jev 입력에서 제외했습니다. GPT와 Gemini는 방식 이름과 상대 결과를 보지 않고 각각 채점했으며, 실행 순서도 번갈아 배정했습니다.

이 조건에서 GPT 기준 근거 총족률은 기존 53/80건(66.25%)에서 Jev 70/80건(87.50%)으로 높아졌고, Gemini도 52/80건에서 74/80건으로 개선을 확인했습니다. 답 없는 질문에 근거를 반환한 경우는 20건에서 1건으로 줄었습니다. 후보 추리기를 포함한 처리시간 중앙값은 기존 0.431초, Jev 9.090초였습니다. 이때 Jev는 묶음을 순차 처리했습니다.

판단

같은 후보와 반환 조건에서도 Jev의 근거 선택이 더 나아, 검색은 후보 수집을 맡고 Jev는 최종 근거 판단을 맡는 구조를 유지하기로 했습니다. 다만 공개 논문과 직접 작성한 질문의 결과이므로 일반적인 정확도로 단정하지 않았습니다. 순차 호출의 대기 시간을 줄이기 위해 다음에는 후보 수와 묶음 처리 방식을 조정하기로 했습니다.

실험과 개선 과정 (계속)

4 300개 후보·4병렬 처리로 대기 시간 단축

문제 상황

Jev로 근거 선택은 개선됐지만, 후보를 최대 500개까지 순차 처리하면 준비와 선택 시간의 중앙값이 9.090초로 기존 방식의 0.431초보다 길어졌습니다. 정확도를 유지하면서 대기 시간을 줄여야 했습니다.

시도와 결과

후보 처리 시간을 줄이기 위해 후보를 상위 300개로 줄이고 최대 4개 묶음을 병렬 처리해 시간은 2.251초로 줄었고, 필요한 근거를 모두 고른 질문은 70개에서 73개로 늘었습니다.

판단

시간을 줄였음에도 2.251초로 기존 0.431초보다는 느렸습니다. 후보를 더 줄이고 더 많은 묶음의 병렬 처리 시도를 고려해봤으나, 필요한 근거가 250위보다 아래에 존재하는 사례가 있어 해당 안은 기각했습니다.

비록 처리 시간이 늘었지만, 정확도가 서비스의 핵심이라고 판단하여 Jev를 채택하기로 결정했습니다.

5 후보 생성 방식 비교로 운영 설정의 한계 확인

문제 상황

Jev가 최종 근거를 고르더라도 필요한 근거가 후보에 없으면 고를 수 없습니다. 운영 중인 후보 생성 방식이 정답 근거를 어디까지 남기는지 확인해야 했습니다.

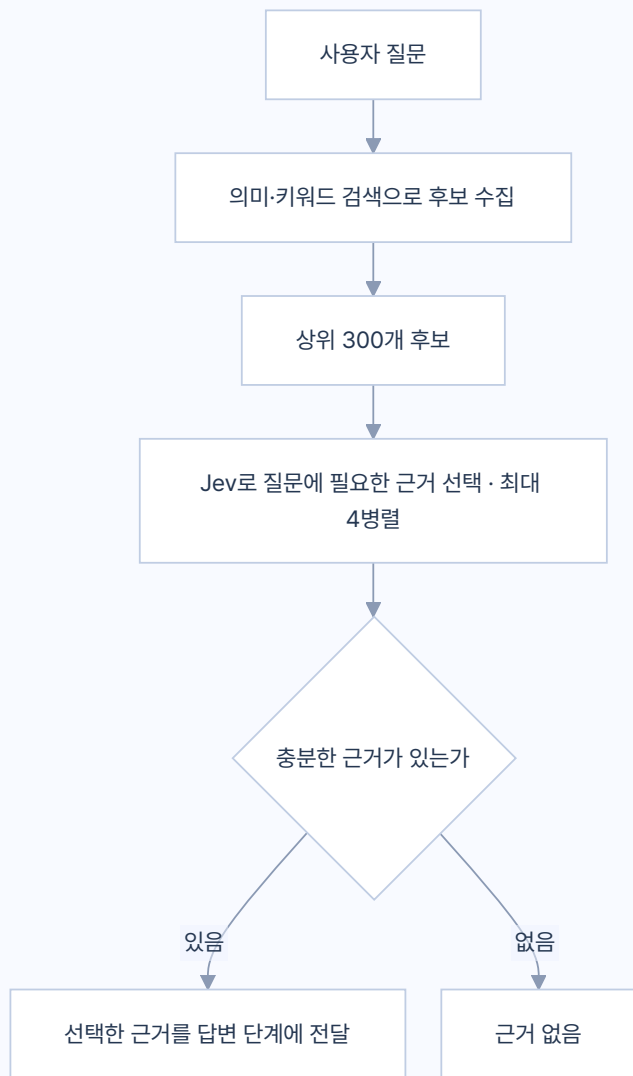
시도와 결과

저장된 점수를 재사용해 후보 생성 네 방식의 순위를 다시 만들고, 정답이 있는 80문항·사전 인용 97묶음의 보존을 비교했습니다. BGE-M3 단독, 운영 중인 혼합(임베딩 0.75 + 정규화 BM25 0.25), RRF, 교대 합집합입니다.

후보 100개에서 정답 인용을 모두 보존한 문항은 BGE-M3 단독 74/80, 운영 혼합 63/80, RRF 63/80, 교대 합집합 65/80이었습니다. 인용별 최선 순위의 MRR@500은 BGE-M3 단독 0.1811, 운영 혼합 0.1245였고 순위 중앙값은 19위와 40위였습니다. 후보를 300개 이상으로 늘리면 네 방식의 차이는 79/80과 78/80으로 좁아졌습니다.

판단

운영 중인 혼합 가중치가 BGE-M3 단독보다 정답 근거를 앞에 두지 못하고 있었습니다. 다만 저장 점수를 재사용한 재현이고 이미 반복해서 쓴 평가셋이므로 독립 검증으로 보지 않았고, 후보를 300개로 두는 현재 구성에서는 차이가 작아 운영 설정을 바로 바꾸지 않았습니다. 후보 수를 더 줄이려면 이 순위 차이를 먼저 해결해야 한다는 조건으로 남겼습니다.



근거 충족 건수 53/80 → 73/80, 총 처리시간 2.251초

근거 충족률(Golden set을 통한 LLM 평가)

66.25% → 91.25%

기존 선택기 53/80 → Jev 개선안 73/80

Jev 근거 선택 시간

중앙값 9.090초 → 2.251초

500후보 순차 → 300후보 4병렬

답 없는 질문 20건에 대한 근거 반환

20건 → 0건

기존 선택기 → Jev 개선안

자체 제작 Golden Set 기준 답이 있는 80개 질문에서 필요한 사실을 모두 뒷받침한 비율이 66.25%에서 91.25%로 높아졌습니다. 별도 20개 질문에서는 답이 없는 문서에 근거를 잘못 붙인 경우가 20건에서 0건으로 줄었습니다. 개선 후에도 기존 선택기보다는 느립니다. 하지만, 정확도가 서비스에 더 중요하다고 판단해 Jev를 채택하기로 결정했습니다. 추가로 같은 뜻의 표현을 묶는 판단에도 적용해, 76문항의 정답 수는 기존 71개·Jev 72개로 비슷하게 유지하면서 판단시간 중앙값을 8.162초에서 0.254초로 줄일 가능성을 확인했습니다.

결과의 범위 · 트레이드오프

시간 단축은 Jev의 설정을 바꾸기 전후의 비교입니다. 검색이 후보 수집을, Jev가 최종 근거 판단을 말도록 단계를 나눠 선택 방식만 교체할 수 있게 했습니다. Jev는 2026년 9월에 공개된 외부 판단 모델이라 장기 운영 근거는 아직 확보하지 않았고, 개선 후에도 기존 선택기보다 느립니다. 답변 생성은 스트리밍으로 전달하지만 근거 선택은 답변 전에 끝나야 하는 단계라 2.251초는 사용자 대기로 남고, 이 구간을 줄이는 일은 과제로 남겼습니다. 후보 생성 비교는 저장된 점수를 재사용한 순위 재현이며 후보 보존율이지 최종 답변 정확도가 아닙니다.

[Jev 공식 소개](#) · [TypeSafe AI](#) ↗

사용자 정의 Agent Skill과 안전한 실행 설계

회의록 생성과 같이 반복하는 문서 작업을 사용자가 직접 정의하고 재사용할 수 있도록 Agent Skill을 설계하고 AI 코딩 도구로 구현했습니다. 작업 지침을 작성하고 검사하는 단계부터 사용자가 검토·게시하고, 실제 변경을 승인하는 단계까지 연결했습니다.

2026.08 - 2026.09

LangGraph

Agent Skill

Prompt Injection 대응

LLM Evaluation

Human-in-the-loop

컨텍스트 엔지니어링

대화 요약 메모리

LangSmith

LLM Observability

배경

회의록 정리처럼 사용자마다 반복하는 문서 작업이 달라, 개발자가 모든 기능을 미리 만들 수 없었습니다. 또 AI가 만든 편집안이 요청 일부를 빠뜨리거나 문장을 어색하게 바꾼 채 사용자 검토 단계까지 전달됐고, 코드의 형식·범위 검사만으로는 이를 걸러낼 수 없었습니다.

기존 Agent

94 / 114

코드 수정 · 평가자 제외

98 / 114

평가자 포함 최종 구현

104 / 114

판단과 구현 과정

1 사용자 정의 작업 지침의 작성·게시 기능 구현

문제 상황

회의록 정리처럼 사용자마다 반복하는 작업이 달라, 개발자가 모든 기능을 미리 추가하는 데는 한계가 있었습니다.

판단

기능을 하나씩 추가하는 대신, 사용자가 자연어로 작업 지침을 정의하고 재사용하도록 하기로 했습니다. AI가 만든 지침은 사용자 검토 전까지 제안으로만 다루기로 했습니다.

구현

자연어 요청을 재사용 가능한 Markdown Skill로 구체화했습니다. 사용자가 초안을 검토·수정하고 최종 게시할 때 Skill과 버전을 저장하도록 구현했습니다. 게시한 Skill은 덮어쓰지 않고 불변 버전으로 덧붙이고, 실행할 버전을 가리키는 포인터를 따로 두었습니다. 어떤 실행이 어느 버전의 지침을 썼는지 추적할 수 있어, 새 버전에 문제가 있어도 이전 정의가 남습니다.

확인한 결과

게시 전에는 Skill이나 임시 초안을 DB에 남기지 않고, 최종 게시한 작업 지침을 이후 작업에서 재사용할 수 있게 했습니다.

2 작업 지침의 권한·승인 우회 방지 검증

문제 상황

사용자가 만든 작업 지침과 참고 문서에 개인정보나 권한·승인을 우회하는 지시가 포함될 수 있었습니다. 같은 의도를 표현하는 방식이 다양했고, 검토한 초안도 게시 전에 사용자가 수정할 수 있었습니다.

판단

명확한 위험과 권한 검사는 코드가 맞고, 표현을 바꾼 우회 의도는 LLM으로 검사하기로 했습니다. 검토 후 사용자가 내용을 수정할 수 있으므로 이전 검사 결과만으로 게시를 허용하지 않기로 했습니다.

구현

입력과 참고 문서에 규칙 검사를 적용하고, LLM이 지적한 위험 문구가 실제 원문에 있는지 서버에서 대조했습니다. 참고 문서는 권한과 원문을 검사한 뒤 필요한 구조만 모델에 전달했습니다. 최종 게시 시에는 수정된 내용으로 검사를 다시 수행했습니다. 위험 문구는 원문에서 자동으로 삭제하지 않고, 사용자가 재생성을 선택한 경우에만 제거한 뒤 결과를 다시 검사하도록 했습니다.

확인한 결과

차단된 입력의 원문과 문제 위치를 사용자에게 보여주도록 했습니다. 사용자는 차단 이유를 보고 직접 수정하거나 재생성을 선택할 수 있게 됐습니다.

판단과 구현 과정 (계속)

3 변경 미리보기와 사용자 승인 후 반영

문제 상황

Skill을 검토해 게시했다라도 실행할 때 바뀔 문서와 내용은 별도로 확인해야 했습니다. 작업 지침의 게시만으로 실제 문서 변경까지 승인한 것으로 볼 수는 없었습니다.

판단

지침 게시와 실제 문서 변경은 별도의 승인으로 나누기로 했습니다. 작업 분류는 라우터가 맡고 실행기는 정해진 작업을 수행하되, 무엇이 바뀌는지는 사용자가 확인하도록 했습니다.

구현

편집은 수정 전후 차이를, 폴더 정리는 만들 폴더와 이동할 문서를 계획으로 보여줬습니다. 출력 형식과 수정 범위는 코드로 검사하고, 승인한 변경만 실행하도록 연결했습니다.

확인한 결과

문서 6개를 사용한 계획·승인·실행 확인에서 승인 전에는 변경이 없었고, 승인 후 주제별 폴더 3개를 만들고 문서를 옮기는 흐름을 확인했습니다.

4 대화 맥락과 편집 입력의 범위 결정

문제 상황

대화가 길어지면 이전 턴을 전부 모델에 넣을 수 없고, 문서 편집에서 문서 전체를 보내면 비용과 지연이 커집니다. 무엇을 남기고 무엇을 자를지 정해야 했습니다.

판단

대화는 최근 여섯 턴을 원문으로 유지하고 그 이전은 누적 요약으로 압축하기로 했습니다. 편집은 문서 전체가 아니라 대상 줄 범위와 주변 문맥만 전달하되, 표나 코드 블록의 경계를 걸치는 범위는 거절하기로 했습니다.

구현

최근 메시지가 여섯 개에 이르면 기존 요약과 최근 메시지를 함께 넣어 누적 요약을 갱신하고, 요약 호출이 실패하면 이전 요약을 유지해 대화가 끊기지 않게 했습니다. 라우터에는 요약과 최근 메시지에 이전 턴의 라우팅 결정까지 함께 전달해, 지시어가 가리키는 대상을 이전 턴에서 찾도록 했습니다. 편집 입력은 대상 줄 범위에 위아래 문맥 줄을 붙여 자르고, 구조 경계를 걸치는 범위는 예외로 막았습니다.

확인한 결과

문서 길이별로 전송한 입력이 원문 대비 어느 정도인지와 처리 시간의 평균·p95를 측정하는 벤치마크를 두고 확인했습니다.

판단과 구현 과정 (계속)

5 LLM 실행 추적과 민감정보 경계 설정

문제 상황

평가 루프가 일반 Python 흐름으로 돌아 의미 추출·평가·보정·재시도를 단계별로 확인할 수 없었습니다. 반대로 추적을 켜면 도구 조회 원문과 개인정보가 외부 추적 도구로 나갈 수 있었습니다.

판단

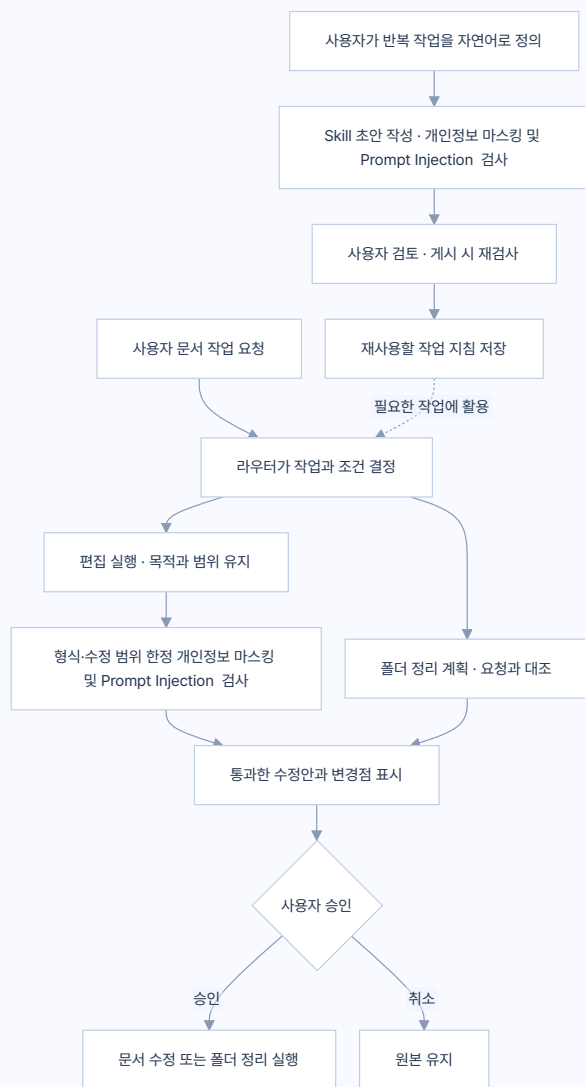
볼 수 있어야 하는 구간과 내보내면 안 되는 내용을 나누기로 했습니다. 응답 품질을 다루는 그래프는 노드 단위로 추적하고, 사용자 문서를 직접 만지는 Agent 그래프는 추적을 끄고 원문을 상태에 남기지 않는 방향입니다.

구현

평가 루프를 LangGraph 그래프로 재구성해 응답 생성·평가·재시도를 각각의 노드로 드러냈습니다. 추적 설정이 켜져 있어도 키가 없으면 그래프 실행은 유지하고 추적만 생략해, 경고와 불필요한 네트워크 재시도를 막았습니다. Agent 그래프는 추적을 끄고 도구 조회 원문을 상태 저장에서 제외했으며, 보관 기간이 끝나면 저장 상태를 실행 기록보다 먼저 삭제하도록 했습니다. 직접 남기는 프롬프트 로그는 기록 전에 숫자 개인정보를 마스킹합니다.

확인한 결과

요청을 실제로 실행해 그래프 노드와 모델 호출 구간이 추적에 나타나는지 확인했습니다. 시간을 재는 실험에서는 추적과 프롬프트 로그를 모두 끈 상태로 실행해, 계속 부담이 측정한 시간에 섞이지 않게 했습니다.



사용자 정의 작업과 승인 흐름 구현 · 편집 품질 82.5% → 91.2% (114건 평가)

기존 Agent

94 / 114

82.5%

코드 수정 · 평가자 제외

98 / 114

86.0%

평가자 포함 최종 구현

104 / 114

91.2%

사용자가 반복 작업을 직접 정의해 Skill로 게시하고 재사용할 수 있게 했습니다. 지침을 게시할 때는 최종 내용을 다시 검사하고, 실제 문서 변경은 미리보기와 사용자 승인을 거치도록 했습니다. 폴더 정리는 문서 6개를 사용한 계획·승인·실행 확인에서, 승인 이후에만 변경되는 것을 확인했습니다. 편집 결과의 품질은 19종 요청에서 얻은 동일 초안 114개로 따로 비교했습니다. 기존 구현은 94개가 통과했습니다. 먼저 요청을 다시 해석하며 생기는 오류를 줄이려고 편집 목적을 편집 단계까지 전달하고 역할별 규칙을 나눴습니다. 구간 편집에서 고정 문구가 중복되던 문제는 조립된 이전 결과와 실패 사유를 재시도에 넘겨 고쳤고, 여기까지 98개가 통과했습니다. 이어 요청 충족과 의미 보존을 별도의 LLM 평가로 검사하고, 실패 사유를 반영해 최대 한 번만 다시 쓰게 해 104개가 통과했습니다. 평가 단계가 생성용 JSON 형식까지 따지며 정상안을 거부해 생성·평가 프롬프트를 분리했고, 제목 추가와 서식 변경을 함께 요청할 때 과하게 막던 보존 검사도 고쳤습니다. 다시 실패하면 수정안을 반환하지 않도록 했습니다. AI는 작업 지침과 수정안을 작성하고, 코드는 권한과 실행 범위를 검사하며, 사용자는 게시와 최종 반영을 결정하도록 책임을 나눴습니다.

결과의 범위 · 트레이드오프

편집 품질 평가의 환산 평균 시간은 11.11초에서 18.92초로 늘었습니다. 지연을 감수하되 재작성 횟수를 제한했습니다.

구현 회고 - 사용자가 직접 만드는 Agent Skill [↗](#) 후속 회고 - 신뢰도를 위한 LLM 평가의 양면성 [↗](#)

CASE 03

Fruition / AI Agent · 보상 복구 · 감사 이력

AI 작업 취소와 이미 저장한 변경의 복구

AI 작업은 모델 호출 사이에 문서와 폴더, Wiki, Skill을 여러 번 저장합니다. 실행만 멈추면 사용자가 취소한 작업의 일부가 남기 때문에, 이미 커밋한 변경까지 역순으로 되돌리는 경로를 만들었습니다. 다른 작업이 남긴 정상 기여는 지우지 않도록 append-only 이력에서 대상 기여만 제외해 복원했습니다.

2026.08 — 2026.09

보상 복구

먹등기

append-only 이력

PostgreSQL 실행 잠금

감사 이력

배경

여러 저장소에 걸쳐 실행되는 AI 작업을 하나의 장시간 트랜잭션으로 묶을 수 없는 상태에서, 사용자가 취소했을 때 무엇을 어디까지 되돌릴지 정해야 했습니다.

복구 대상 저장소

3곳

복구 실행

**모델 호출
없음**

취소 확정 조건

**전체 복구
확인**

판단과 구현 과정

1 실행 중단만으로는 부족하다는 판단

문제 상황

AI 작업은 모델 호출 사이에 문서와 폴더, Wiki, Skill을 여러 번 저장합니다. 실행만 멈추면 사용자가 취소한 작업의 일부가 그대로 남았습니다. core DB와 AI DB, 객체 저장소에 걸친 실행을 하나의 장시간 트랜잭션으로 묶을 수도 없었습니다.

판단

이미 커밋한 변경까지 되돌리기로 했습니다. 모델에 반대 작업을 요청하는 방법은 원래 값과 승인 범위, 역등성을 보장할 수 없어 기각했고, 서비스 전체를 단일 트랜잭션으로 잠그는 방법은 모델과 HTTP, 객체 저장소의 장시간 실행과 장애를 감당할 수 없어 기각했습니다.

구현

AI 쪽이 실행 기록으로 역작업을 정하고, 업무 데이터의 원자적 복구는 backend가 수행하도록 역할을 나눴습니다. Agent 도구는 변경 전 snapshot과 원래 승인에 연결된 역작업 인자·역등키를 저장하고, 일반 작업은 각 DB의 trigger로 변경 전후 값을 같은 트랜잭션에 기록했습니다. 객체 저장소는 쓰기 전에 원본과 예정 결과를 남겼습니다.

확인한 결과

복구가 모델 설정이나 호출 없이 실행 기록만으로 수행되는지 확인했습니다.

2 취소 이후의 쓰기를 막고 자식부터 역순으로 복구

문제 상황

취소를 받아도 실행 중인 handler가 계속 쓰면 복구한 값이 다시 덮였습니다. 부모 작업을 먼저 되돌리면 자식이 남긴 변경이 정리되지 않고 남았습니다.

판단

취소 상태를 먼저 저장해 후속 쓰기를 막고, 실행 중인 handler가 끝난 뒤 자식부터 역순으로 되돌리기로 했습니다.

구현

Agent 턴과 복구가 같은 PostgreSQL 실행 잠금을 쓰도록 했습니다. worker 연결이 끊기면 잠금이 해제되므로, 남아 있는 실행 중 기록이 복구를 막지 않습니다. 본문 복구는 새 revision으로 저장해 늦게 도착한 이벤트가 원문을 다시 덮어쓰지 못하게 했습니다.

확인한 결과

전체 복구를 확인해야 취소가 확정되도록 하고, 충돌이나 불명확한 결과는 재시도 상태로 남겼습니다.

판단과 구현 과정 (계속)

3 다른 작업의 기여를 지우지 않는 복구

문제 상황

Wiki의 개념 페이지는 여러 문서의 기여를 합친 결과입니다. 특정 작업을 과거 snapshot으로 덮어쓰면 이후 추가된 다른 문서의 정상 기여까지 사라졌습니다.

판단

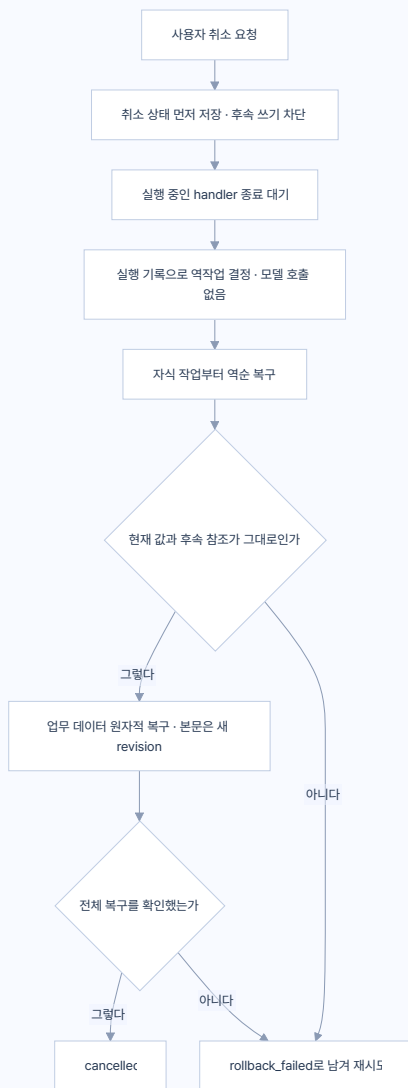
완성된 결과에서 해당 부분만 지우는 방법은 어떤 문장이 어느 문서에서 왔는지 안정적으로 구분할 수 없어 기각했고, 과거 snapshot으로 전체를 되돌리는 방법도 기각했습니다. 이력을 덧붙이기만 하고 복구할 때 대상 기여만 제외해 다시 조립하기로 했습니다.

구현

작업과 페이지 revision, 문서별 기여를 덧붙이기만 하는 이력으로 기록하고, 복구할 때 대상 기여를 제외한 상태로 복원하거나 재조립했습니다. 페이지와 이력은 물리 삭제하지 않고 삭제 표시만 남겨 감사 이력을 보존했습니다.

확인한 결과

비동기 복구 중에 상태가 바뀌지 않았는지 revision과 기여 hash로 검증했습니다. 복구 자체도 새 작업과 revision으로 기록해 무엇을 되돌렸는지 남겼습니다.



취소한 작업이 남긴 변경을 되돌리고, 다른 작업의 기여는 유지

복구 대상 저장소

3곳

core DB · AI DB · 객체 저장소

복구 실행

모델 호출 없음

실행 기록의 역작업만 사용

취소 확정 조건

전체 복구 확인

미확인은 재시도 상태로 보존

사용자에게 보이는 내용과 이름, 부모, 순서는 복원하고 버전과 감사, 역등 기록은 남기도록 했습니다. 복구는 모델 설정이나 호출 없이 실행 기록만으로 수행하며, 현재 값과 후속 참조를 확인해 다른 작업의 변경을 덮어쓰지 않습니다. 전체 복구를 확인해야 취소가 확정되고, 충돌이나 불명확한 결과는 재시도 상태로 남깁니다. AI가 문서를 바꾸는 기능에서 되돌릴 수 있는 범위와 남겨야 할 기록을 먼저 정하고 구현한 경험입니다.

결과의 범위 · 트레이드오프

설계와 단위 검증 중심의 사례이며, 운영 트래픽에서의 복구 성공률은 측정하지 않았습니다. Ingest 시점부터 Source별 기여를 보존해야 하므로 저장 비용이 늘어납니다. 남은 기여 조합의 snapshot이 없으면 Page를 다시 조립해야 하고, snapshot이 없는 구버전 작업 때문에 배포 전에 실행과 큐를 비워야 했습니다.

PDF → Markdown 변환 - 내부 평가 통과율 45.17% → 89.89% 개선

주로 PDF로 제공되는 논문을 지식으로 활용하려면, Markdown으로 변환 시 표의 수치와 수식이 먼저 보존돼야 했습니다. 이를 위해 변환 결과를 AI로 고치는 과정에서 생기는 누락을 추적하고, 원본 배치와 내용 유형에 맞춰 처리 경로를 나눴습니다.

2026.07 — 2026.09

PDF 변환 오픈 소스 : AnyDoc, Docling

LLM API

sLLM (Qwen2.5 7B)

Python

LLM-as-a-Judge

배경

논문 PDF를 서비스의 지식으로 활용하려면 본문뿐 아니라 표·수식·그림도 보존한 Markdown이 필요했습니다.

기존 Docling

45.17%

개선 파이프라인

89.89%

실험과 개선 과정

1 손상 영역 선택 복원과 탐지 누락 확인

문제 상황

오픈소스로 PDF를 Markdown으로 변환하자 표의 행·열 관계와 수식이 손상됐습니다. 전체를 다시 처리하지 않고 잘못 변환된 부분만 복원할 방법이 필요했습니다.

시도와 결과

Docling으로 구조를 뽑고 손상된 표·수식만 골라 로컬 Qwen sLLM으로 복원했습니다. 행·열 위치가 중요한 표는 비전 모델(qwen2.5vl:7b) 중심으로, 수식은 텍스트 모델(qwen2.5:7b)로 후보를 만든 뒤 필요할 때만 비전 모델로 확인하게 나눴습니다. 두 모델을 모두 적용한 경우보다 복원율이 높았고 sLLM 처리 시간은 약 42% 줄었습니다. OCR과 프롬프트는 그대로 두고 이미지 crop만 Docling의 엄격한 bbox로 다시 만들자 실패하던 두 수식이 모두 정확히 복원되기도 했습니다.

복원한 부분과 그대로 둔 부분을 각각 평가하자 sLLM이 복원한 96개 중 85개는 통과했습니다. 하지만 복원하지 않은 부분을 확인하자 오류·누락이 107개 남아 있었습니다. OCR 대조를 추가한 별도 실험에서는 통과율을 80.00%에서 97.53%로 높였지만, 정상인 내용 50개도 수정 대상으로 골랐고 처리 단계의 시간 합은 약 12분 35초였습니다.

판단

무엇을 복원할지 고르는 과정에서 오류를 놓치고, 이를 보완하는 검사에도 비용이 들었습니다. 손상 탐지에 의존하게 되고, 그에 따른 비용이 드니 원본 페이지 전체를 비전 sLLM이 읽게 하는게 더 나을 수도 있겠다고 판단했습니다.

2 전체 페이지 sLLM 변환의 표·수식 오류와 비용 확인

문제 상황

손상된 부분만 고르는 방식은 오류를 놓쳤고, OCR 대조를 추가하면 정상 내용도 수정 대상으로 선택하면서 검사 시간과 비용이 늘었습니다.

시도와 결과

논문 PDF 25개(415쪽)를 원본을 직접 보고 확인할 수 있는 비전 sLLM(qwen2.5vl:7b)으로 전체 처리했을 때 내용 보존 통과율은 91.92%였습니다. 하지만 표·수식의 값과 구조가 모두 맞은 비율은 58.84%였고, 모든 페이지를 처리하는 처리 부담도 컸습니다.

판단

모든 내용을 같은 방식으로 모델에 맡기기보다, 본문은 필요한 부분에만 AI를 쓰고 표·수식은 각각의 구조를 보존할 수 있는 입력을 제공해야겠다고 판단했습니다. 본문과 표, 수식의 처리 경로를 나누고, 본문을 빠르게 추출하면서 필요한 부분만 AI로 복원할 방법을 모색했습니다.

실험과 개선 과정 (계속)

3 본문 추출 비교, AnyDoc 적용과 상용 API 전환

문제 상황

페이지 전체를 sLLM으로 처리해도 표·수식 오류가 남았고, 모든 본문까지 모델에 보내는 부담이 컸습니다. 표·수식은 별도로 복원하면서 본문은 빠르게 추출할 방법이 필요했습니다.

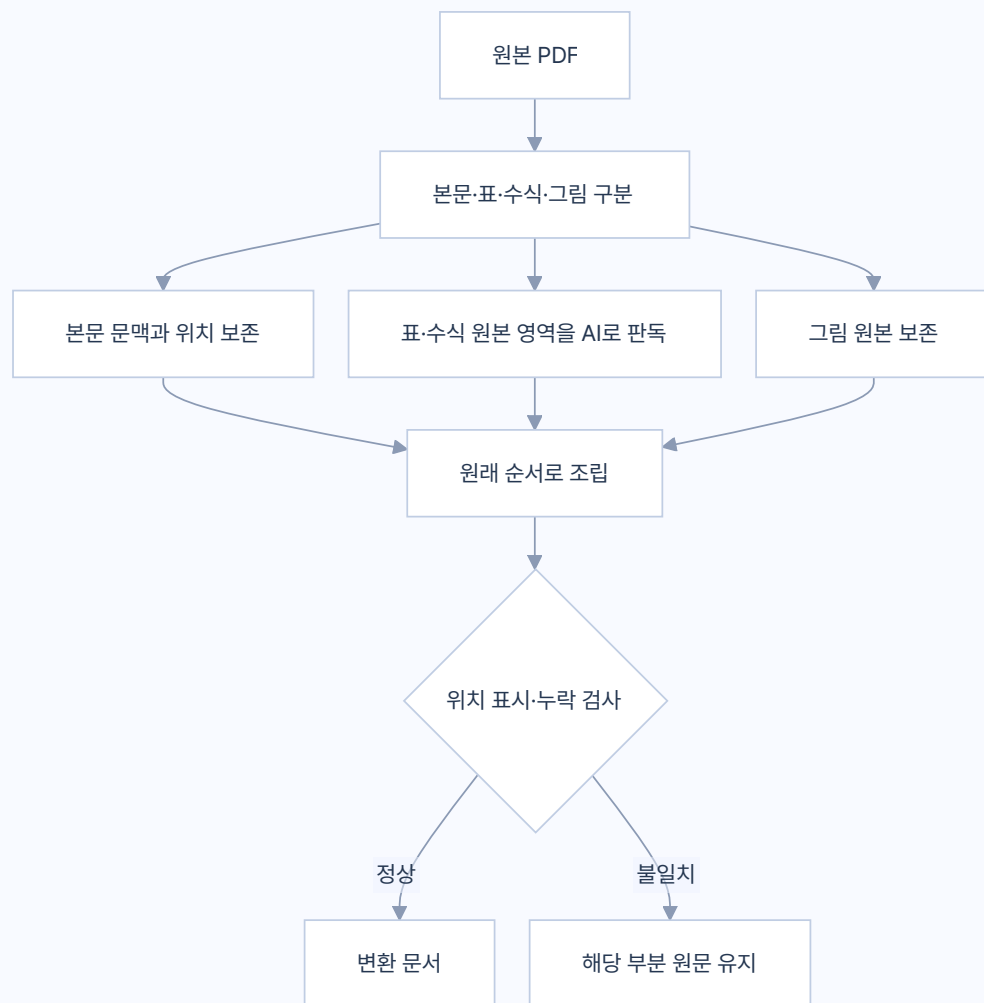
시도와 결과

Threads에서 PDF 변환 도구 AnyDoc을 알게 됐고, GitHub의 Docling 대비 평가 결과를 보고 기존 실험에 쓰던 PDF 4개(30쪽)로 비교했습니다.

기존 Docling 처리 기록은 134.19초였고, AnyDoc의 텍스트 추출은 1.68초로 약 80배 짧았습니다. Docling은 OCR과 표 분석까지 포함한 시간이었습니다. AnyDoc 결과에서는 한 문서의 글자가 깨졌고, PDF 내부에 글자 코드를 실제 문자로 연결하는 정보가 빠진 것이 원인이었습니다.

판단

처리 범위가 달라 전체 변환이 80배 빨라졌다고 볼 수는 없지만, 본문을 빠르게 추출하는 데 활용할 가치가 있다고 판단했습니다. 표와 수식은 이미 별도로 복원하고 있었으므로 그 과정은 유지하고, 본문 추출을 AnyDoc으로 바꾸되 글자가 깨진 페이지는 AI로 보완하기로 했습니다. 읽는 순서가 섞인 페이지도 AI로 복원하고, 그림은 원본을 보존했습니다. 또 초기 사용자가 없는 단계에서 GPU 서버를 띄워 sLLM을 운영하는 비용이 더 크다고 판단해, 이때부터 복원에 쓰는 모델을 상용 LLM API로 바꿨습니다.



30페이지 원문 대조에서 실사용 기준 통과율 45.17% → 89.89%

기존 Docling

45.17%

총 30페이지

개선 파이프라인

89.89%

총 30페이지

같은 논문의 본문·표·수식 등을 445개 평가 단위로 나눠 원문과 비교했습니다. 기존 Docling 변환과 개선한 파이프라인의 결과를 이름을 숨긴 상태로 평가했습니다. 실사용 기준 통과율은 201개(45.17%)에서 400개(89.89%)로, 값·기호까지 완전히 같아야 하는 엄격 기준은 193개(43.37%)에서 393개(88.31%)로 높아졌습니다.

결과의 범위 · 트레이드오프

PDF 4개(30쪽)에 대한 내부 LLM 평가입니다. 본문, 제목, 그림은 경미한 차이를 허용했고, 표와 수식은 값, 기호, 구조가 일치해야 통과시켰습니다.

[PDF → Markdown에 SLLM 사용해보기 ↗](#) [PDF 변환 오픈소스 AnyDoc을 사용해보자 ↗](#)

CASE 05

Fruition / Kafka 비동기 병렬 처리 · AI 파이프라인

문서 병렬 분석 - 처리시간 282.11초 → 73.89초 단축

문서를 활용한 Wiki 생성이 순차처리로 구성되어 있어, 대형 문서의 Wiki 생성 시 처리 시간이 증가해 사용자의 부정적 경험을 우려했습니다. 이에 Worker를 늘린 효과와 자원 사용을 확인하고, 문서 분석과 벡터 생성에 서로 다른 확장 기준을 세웠습니다.

2026.09

AI 파이프라인

Kafka

PostgreSQL

동시성·메모리

배경

문서를 순차 처리하면 뒤의 문서가 앞선 문서의 외부 AI 응답을 기다려야 했습니다. Worker를 1개에서 4개로 늘려 분석 시간을 282.11초에서 73.89초로 줄인 뒤, 동시성을 더 높이면 계속 빨라지는지 확인했습니다. Worker 내부의 동시 AI 요청을 2개에서 4개로 늘리자 95.92초로 느려지고, 같은 GPU에서 벡터 생성을 2개 실행하면 각각 약 9.5초에서 19~21초로 늘었습니다. 외부 응답을 기다리는 분석과 장치를 나눠 쓰는 벡터 생성은 병렬화 효과가 달라, 단계별로 확장 기준을 나눴습니다.

Worker 1개

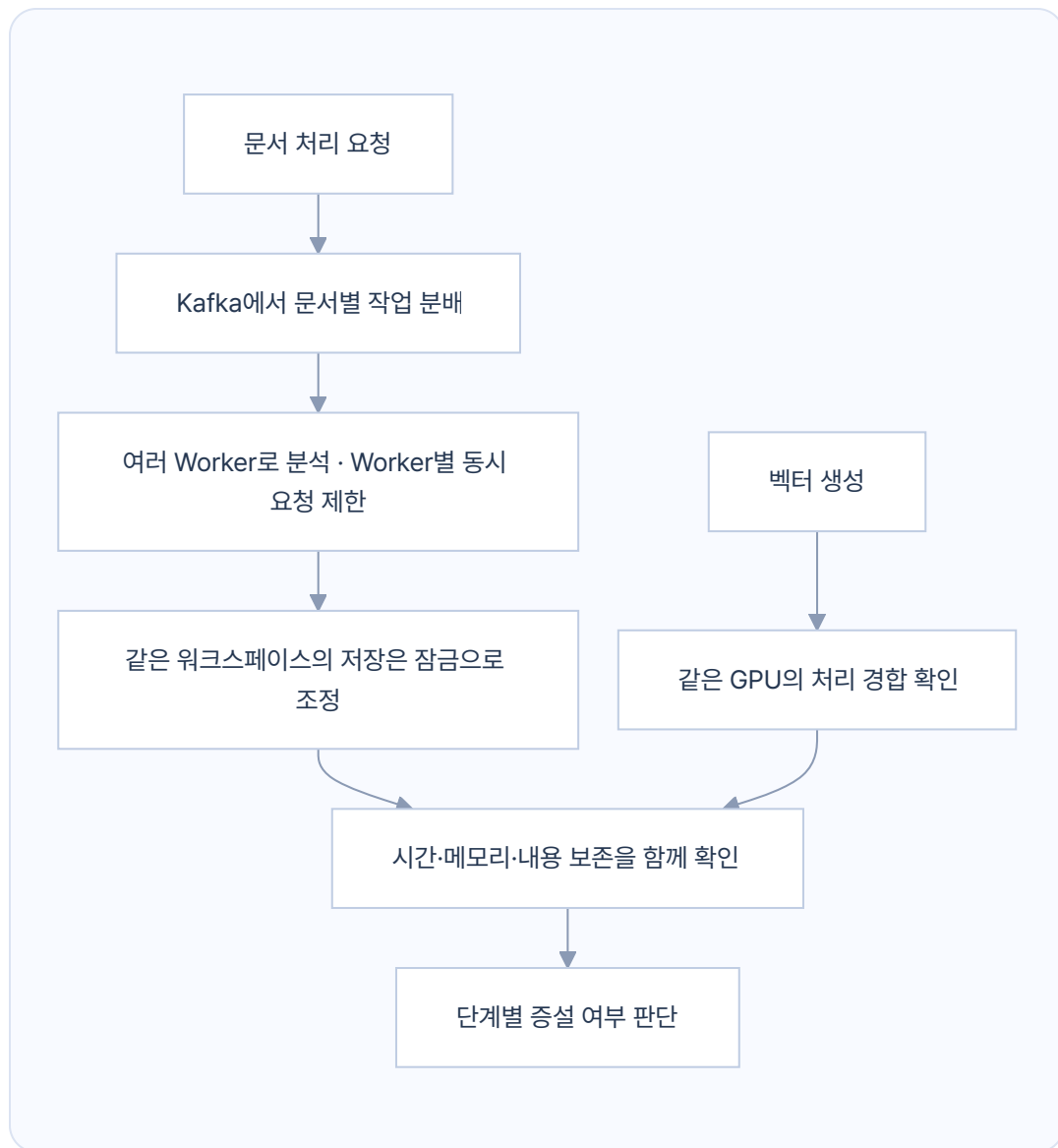
282.11초

Worker 4개

73.89초

프로세스 메모리 합

약 4배



문서 4개 분석 시간 73.8% 단축 · 문서 100개 큐 재현 300건 누락·실패 0건

Worker 1개

282.11초

4문서(8번의 호출)

Worker 4개

73.89초

최대 API 동시 요청 8개

프로세스 메모리 합

약 4배

351.7 → 1,401.0MiB

Worker 4개, 각 2개 동시 요청 조건에서 문서 4개의 총 처리 시간을 약 73.8% 줄였습니다. 동시에 프로세스 메모리 합이 351.7MiB에서 1,401.0MiB로 약 4배 늘었습니다. 미리 정해 둔 사실 16개 중 완전히 보존된 수도 Worker 1·2·4개에서 10·14·11개로 달라, 조건별 1회의 작은 표본이지만 품질이 같다고 보지는 않았습니다. 동시 요청을 더 늘리는 안과 같은 GPU에서 벡터 생성을 병렬화하는 안은 오히려 느려져 기각했습니다. 동시성을 높이면 항상 빨라질 것이라 본 처음 가정이 맞지 않았고, 외부 응답 대기과 장치 경합을 다른 문제로 나눠 판단하게 됐습니다. 저장된 AI 응답으로 문서 100개의 큐 처리를 3가지 조건에서 재현한 결과, 총 300건 모두 누락·실패 없이 처리됐습니다. 메모리를 약 4배 더 사용하지만 MSA로 EKS에서 AI 기능을 제공해 분산이 가능하다고 판단해, 문서 병렬 처리 시 최대 Worker 4개를 채택했습니다.

결과의 범위 · 트레이드오프

같은 문서에 대한 조건별 1회 분석 결과입니다. 저장, 후처리까지 포함한 E2E 기준 속도가 아닙니다.

코딩 에이전트 루프 - MSA 이식 PR 16개를 5일 만에 병합

AI 기능을 MSA 구조로 옮기는 작업을 코딩 에이전트에게 맡기고, 저는 작업 계획과 검토 기준을 정하고 병합 여부를 판단했습니다. 브랜치별 작업 공간마다 에이전트를 나눠 병렬로 실행하고, 조정 에이전트가 순서와 우선순위를 관리하는 루프를 구성했습니다.

2026.08

코딩 에이전트

Claude Code

MSA

코드 리뷰

배경

MSA 전환 마감까지 약 일주일만 남은 상황에서, AI 기능을 새 서비스 구조로 옮기는 작업이 여러 갈래로 나뉘어 있었습니다. 한 세션에서 오래 작업하면 맥락이 요약되면서 앞서 정한 계획을 놓치는 문제가 있었습니다.

병합 PR

16개

병합 전 필수 수정

13건

브랜치당 에이전트

2개

루프 설계와 판단 과정

1 역할을 나눈 에이전트 루프 구성

문제 상황

한 세션에 모든 작업을 맡기면 맥락이 요약되면서 계획과 기준이 흐려졌습니다.

시도와 결과

작업 계획과 기준을 문서로 고정하고, 구현·검토·통합 리뷰로 역할을 나눴습니다. 브랜치마다 작업 공간을 따로 두고 에이전트 2개를 병렬로 실행했으며, 조정 에이전트가 작업 간 의존성과 우선순위를 판단하도록 했습니다.

판단

세션이 바뀌어도 같은 문서를 기준으로 이어갈 수 있어, 긴 작업을 여러 에이전트에 나눠도 방향이 흩어지지 않았습니다.

2 PR 진행 기간: 8/9 → 8/13

문제 상황

작업들이 서로 의존해, 모두 동시에 열면 충돌과 재작업이 생길 수 있었습니다.

시도와 결과

8/9에 #156~#159를 만들어 8/10에 병합하고, 8/11에 #167~#169~#170 등을 병렬로 진행한 뒤, 8/13 #175에서 통합 결함을 고쳤습니다. 5일 동안 PR 16개를 병합했습니다.

판단

선행 작업이 병합된 뒤 다음 묶음을 여는 순서로 진행해, 병렬로 실행하면서도 충돌을 줄였습니다.

루프 설계와 판단 과정 (계속)

3 병합 전 리뷰와 병합 판단

문제 상황

에이전트가 만든 변경을 그대로 병합하면 결함이 함께 들어갈 위험이 있었습니다.

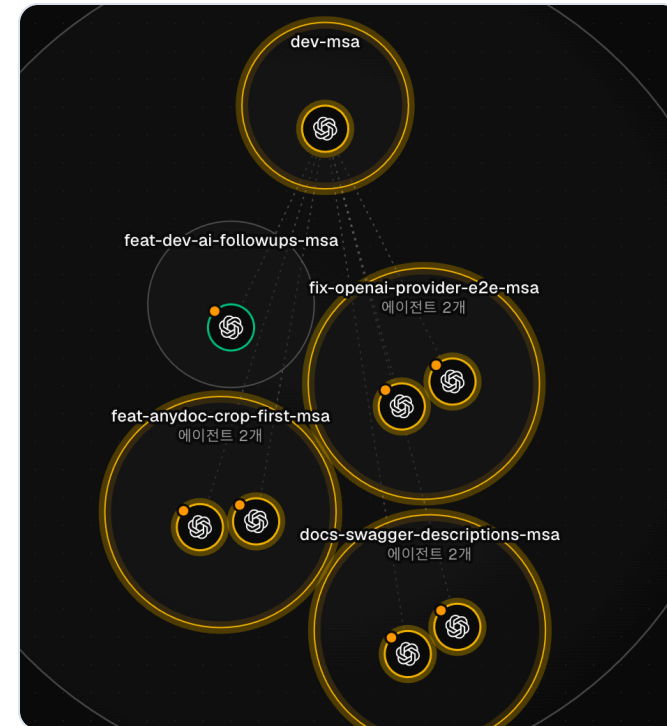
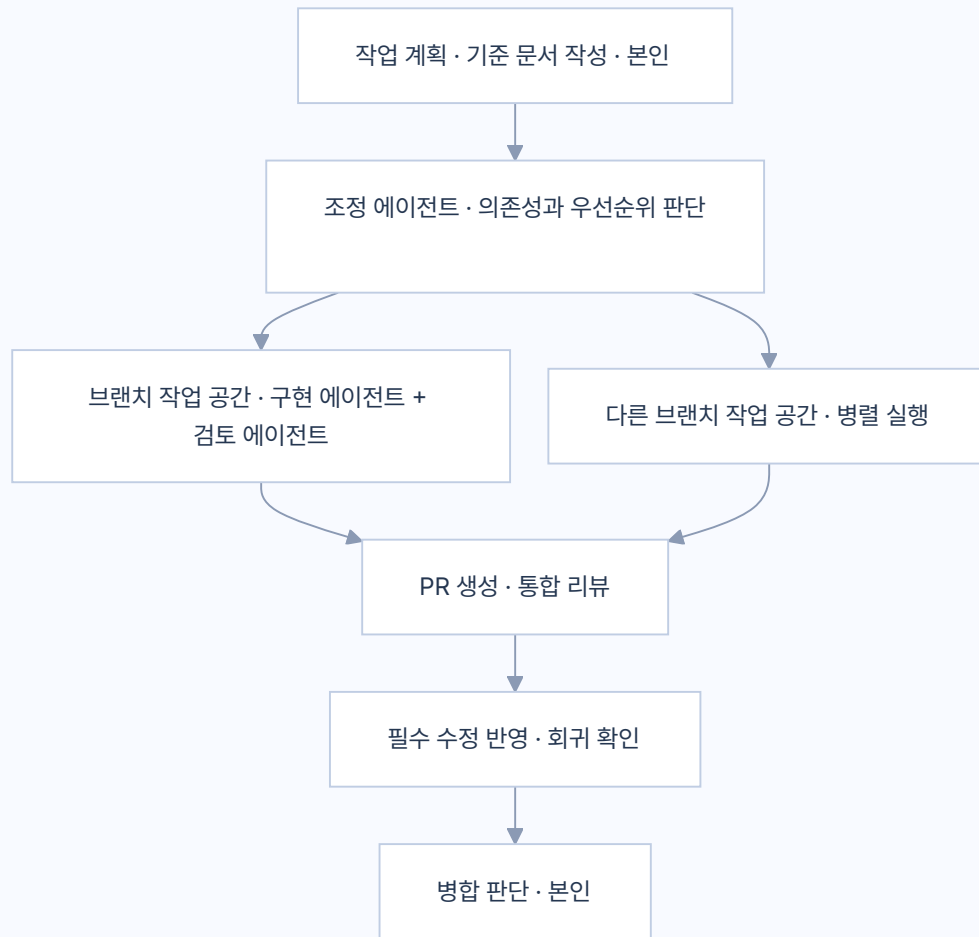
시도와 결과

리뷰 결과 중 필수 수정 13건을 반영하고 회귀를 확인한 뒤에만 병합했습니다. 예를 들어 문서를 복구한 뒤 이전 본문이 반환되던 결함을 병합 전에 고쳤습니다.

판단

코드를 직접 작성하지 않는 대신, 통과 기준을 정하고 그 기준으로 병합을 판단하는 것을 제 역할로 삼았습니다.

개선한 처리 흐름



계획과 기준은 제가 정하고, 구현과 검토는 역할을 나눈 에이전트가 브랜치별로 병렬 수행한 구조입니다.

5일 동안 PR 16개 병합 · 병합 전 필수 수정 13건 반영

8월 9일부터 13일까지 5일 동안 PR 16개를 병합했습니다. 병합 전 리뷰에서 나온 필수 수정 13건을 반영하고 회귀를 확인했습니다. 리뷰가 잡은 결함 중에는 문서를 복구한 뒤 이전 본문이 반환되던 문제가 있었습니다. 구현은 코딩 에이전트가 맡았고, 저는 루프 구성, 작업 계획과 기준 문서 작성, 결과 검토와 병합 판단을 맡았습니다.

결과의 범위 · 트레이드오프

코드를 직접 작성하지 않고 기준 문서와 리뷰 결과로 병합을 판단했습니다. 그래서 리뷰 기준에 없는 결함은 놓칠 수 있어, 기준 문서를 계속 보완하는 것이 과제로 남았습니다.

병합 PR

16개

2026.08.09 – 08.13

병합 전 필수 수정

13건

리뷰 지적 반영

브랜치당 에이전트

2개

구현 · 검토

Pilltip

의약품 정보를 쉬운 말로 안내하고, 사용자·가족의 복약 정보와 일정 관리를 돕는 개인 맞춤 AI 복약 서비스입니다.

시작 계기

부산대학교 졸업과제로 시작했습니다. 약품 정보가 여러 곳에 흩어져 있어, 지금 건강 상태에서 특정 약을 먹어도 되는지 쉽게 확인하기 어려운 문제를 풀고자 했습니다.

GitHub 저장소 [↗](#)

팀 구성과 내 역할

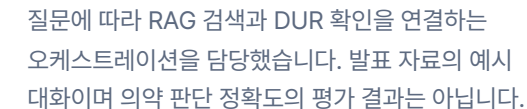
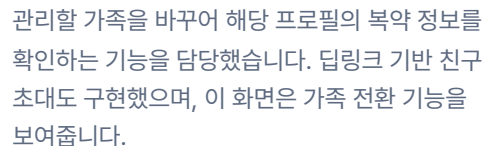
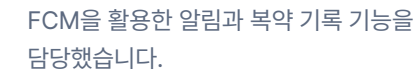
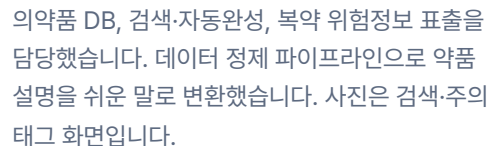
MY ROLE

백엔드·AI

의약품·건강기능식품 DB, 검색·자동완성, 복약 알림, 복용 위험·주의 정보 표출, 데이터 정제, AI 챗봇

- | | |
|---|---|
| 01 백엔드·AI 기능 개발 | 02 의약품·건강기능식품 데이터 모델링, 검색·자동완성과 복용 위험 정보 표출 |
| 03 복약 알림과 기록 | 04 친구, 가족 기능 |
| 05 데이터 정제 파이프라인 구축 - 예상 \$200 대비 실제 \$11.18로 변환 | 06 RAG 기반 AI 챗봇 |
| 07 전시 피드백을 반영한 건강기능식품 지원 확대 | |

Android 1명	Kotlin 기반 Android 앱 전반
백엔드·보안 1명	사용자 DB, 로그인, 문진표, AES-GCM 암호화
디자인	디자이너



의약품 정제 파이프라인 - 예상 \$200 대비 실제 \$11.18로 변환

어려운 의약품 정보를 이해하기 쉬운 표현으로 바꾸기 위해 데이터 정제 파이프라인을 구축했습니다. 원문을 문장 블록으로 나눠 같은 문장은 한 번만 변환하고, 결과를 각 약품에 다시 연결해 실제 API 지출 \$11.18로 처리했습니다. 2026년에는 같은 정제 과정을 PySpark로 재현해 단계별 감소 효과를 측정했습니다.

2025.07 · 재현 2026.10

데이터 파이프라인

GPT Batch API

PySpark

MinHashLSH

배경

약 4만 4천 건의 약품 원문을 사용자가 이해하기 쉬운 설명으로 변환하는 기능을 만들었습니다.

원문 처리 예상 비용

약 \$200

실제 API 지출

\$11.18

완전 일치 중복 제거

-94.1%

문제 분석과 구현 과정

1 완전 일치 문장 중복 제거

문제 상황

약 4만 4천 건, 1GB 규모의 원문을 모두 변환하면 API 비용이 약 \$200으로 예상됐습니다. 예산 안에서 전체 약품의 설명을 제공할 방법이 필요했습니다.

시도와 결과

약품 원문 100개를 조사해 약 70%가 중복되는 내용임을 확인했고, 표본 분석과 작업 계획으로 전처리 기간 7일을 확보했습니다. 원문을 문장 블록으로 나눠 완전히 일치하는 문장은 하나로 통합하고, 각 약품과 문장 블록의 연결을 남겼습니다.

판단

약품 수를 줄이는 대신 같은 문장을 한 번만 변환해 재사용하기로 했습니다.

2 GPT Batch API 일괄 변환과 약품별 재연결

문제 상황

변환 비용을 최대한 줄일 필요가 있었고, 중복은 제거했으나, 모델 비용도 줄일 수 있는 방법을 모색해야 했습니다.

시도와 결과

서비스 사용 중 즉시 응답해야 하는 작업이 아니므로 일괄 처리의 대기 시간을 허용했습니다. 고유 문장을 GPT Batch API로 변환한 뒤, 보존한 약품 ID와 문장 블록 ID의 연결을 이용해 각 약품에 결과를 다시 연결했습니다. 전체 원문 변환에 예상했던 약 \$200 대비 실제 API 지출 \$11.18로 변환을 마쳤습니다.

판단

중복 제거와 Batch 변환을 함께 사용해 약품의 범위를 줄이지 않고 기능을 구현했습니다.

문제 분석과 구현 과정 (계속)

3 2026 PySpark 재현 - 어미 통일과 유사 문장 통합의 효과 측정

문제 상황

2025년 정제 코드 일부와 원본 파일이 남아 있지 않아, 완전 일치 이후 단계의 효과를 수치로 설명할 수 없었습니다.

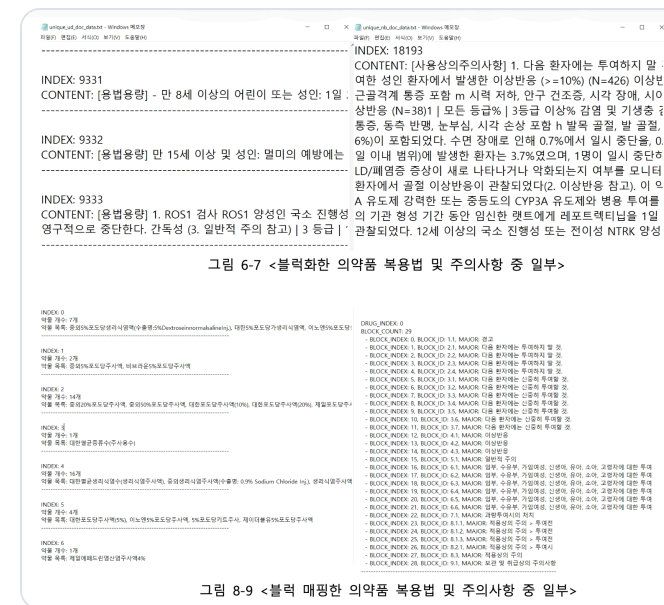
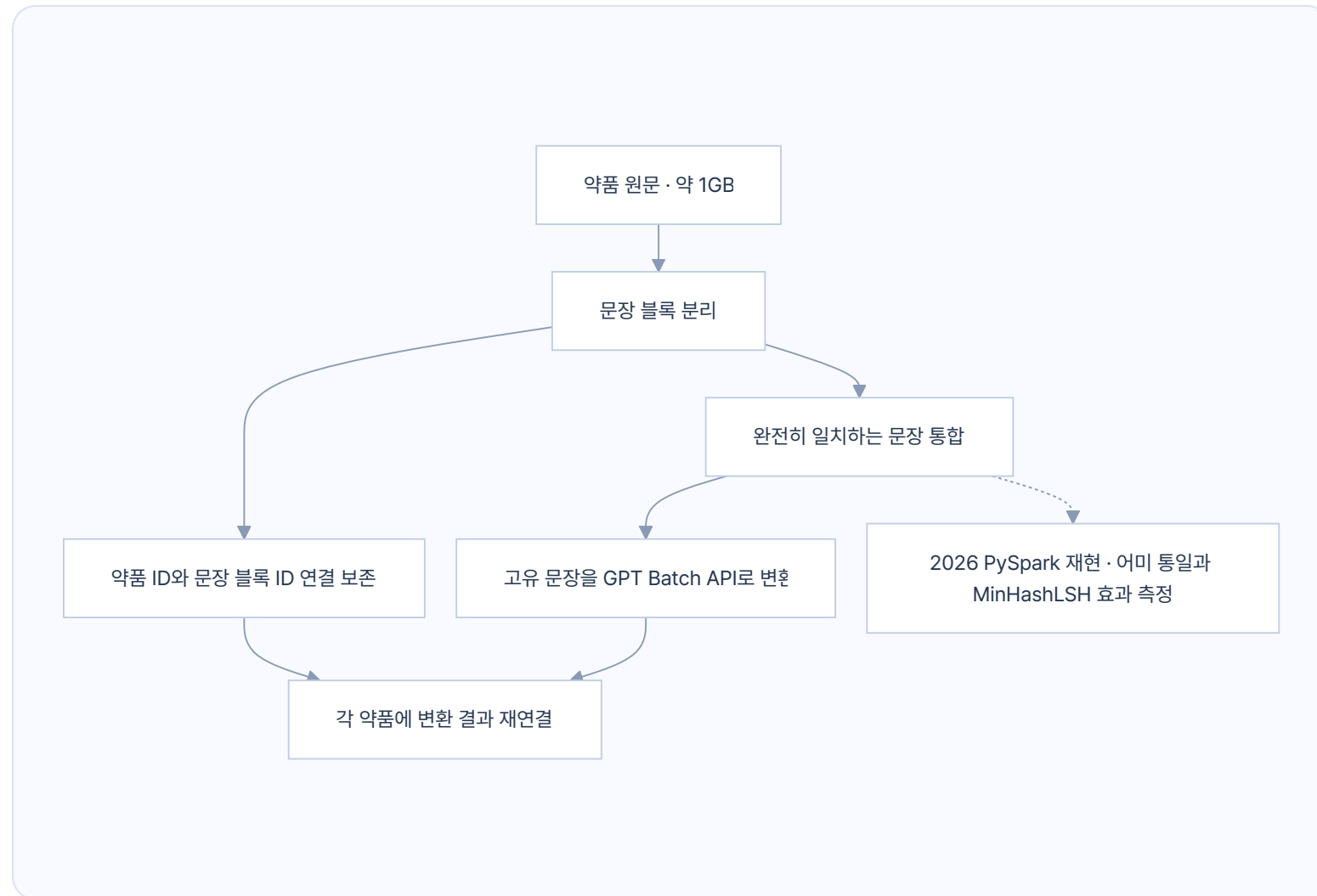
시도와 결과

식약처 공개 API로 42,714개 품목을 다시 수집해 XML 구조를 따라 블록 3,299,602개로 나누고, PySpark로 정규화와 완전 일치 중복 제거를 재현했습니다. 이어 어미 통일 사전과 MinHashLSH(글자 3-gram, Jaccard 거리)를 적용하고, 임계값 0.1·0.2·0.3의 감소율과 합쳐진 표본을 비교했습니다.

판단

0.3에서는 18.78%가 줄었지만 다른 질환(Lapp·Total 유당분해효소 결핍증)의 문장이 합쳐졌고, 0.2에서도 주의 문장 일부가 사라졌습니다. 의료 정보는 잘못 합치는 비용이 커서, 숫자와 적용 조건이 같을 때만 합치는 안전장치와 함께 0.1(추가 6.69% 감소)을 택했습니다.

개선한 처리 흐름



같은 문장은 한 번만 변환하고 약품별 연결을 보존해 결과를 다시 붙였습니다. 2026년에는 같은 과정을 PySpark로 재현해 측정했습니다.

실제 API 지출 \$11.18 · PySpark 재현에서 텍스트 94.1% 감소

전체 원문을 처리할 때 예상한 약 \$200 대비 실제 API 지출 \$11.18로 변환을 마쳤습니다. 설명을 제공할 약품의 범위를 줄이지 않고, 고유 문장의 변환 결과를 재사용해 예산 안에서 기능을 구현했습니다. 2025년 정제 코드 일부가 남아 있지 않아, 2026년에 식약처 공개 API로 42,714개 품목(순수 텍스트 960MB)을 다시 수집하고 같은 정제 과정을 PySpark로 재현했습니다. 블록 3,236,626개가 고유 블록 176,648개로 줄어 텍스트가 744MB에서 44.1MB로 94.1% 감소했습니다. 이어 어미 통일 사전으로 0.78%, MinHashLSH 유사 문장 통합으로 추가 6.69%를 줄였습니다.

결과의 범위 · 트레이드오프

약 \$200은 사전 추산이고 \$11.18은 실제 지출입니다. 2026년 수치는 다시 수집한 데이터로 측정해 2025년 처리 데이터와 같지 않습니다. 재현 코드는 AI 코딩 도구로 작성했고, 실험 설계와 임계값 판단을 맡았습니다.

원문 처리 예상 비용

약 \$200

추산치

실제 API 지출

\$11.18

2025 변환 작업

완전 일치 중복 제거

-94.1%

2026 PySpark 재현 · 744MB → 44.1MB

복약 챗봇 - 건강 프로필을 외부 AI 입력에서 분리

RAG 기반 AI 오케스트레이션으로 복약 관련 정보를 안내하는 AI 챗봇을 앱에 탑재할 수 있도록 했습니다. 사용자 증상에 맞는 의약품 탐색과 프로필에 따른 판단을 나눴습니다. 모델의 언어 해석 능력을 활용하면서 민감한 프로필을 사용하는 범위는 코드로 관리했습니다.

2025.09

Spring AI

Weaviate

RAG

AI Orchestration

배경

사용자가 자연어로 증상을 질문하면 약품 정보를 찾고, 개인 건강 프로필에 맞는 정보를 제공하는 복약 챗봇을 구현했습니다.

설계와 구현 과정

1 증상과 약품 효능을 연결하는 의미 검색 구현

문제 상황

사용자가 말하는 증상과 약품 효능에 쓰인 단어는 같은 뜻이어도 달랐습니다. 단어가 일치하지 않아도 관련 정보를 연결할 방법이 필요했습니다.

판단

자연어 증상과 약품 효능을 연결하는 일에는 의미 검색을 사용하고, 질문에 필요한 정보를 제공하도록 처리 경로를 나누기로 했습니다.

구현

약품 효능을 Weaviate에 저장해 증상과 의미가 가까운 정보를 찾도록 했습니다. 질문에 따라 약품 탐색·복약 위험·섭취량 정보를 처리하는 경로를 구성했습니다.

확인한 결과

증상을 자연어로 입력해 약품 정보를 탐색하고, 질문에 맞는 처리 경로로 연결하는 기능을 구현했습니다.

2 건강 프로필의 내부 처리와 외부 AI 입력 분리

문제 상황

개인화에는 임신 여부·복용약·기저질환이 필요했지만, 이 프로필까지 외부 LLM에 전달하면 민감정보의 유출이 우려되었습니다.

판단

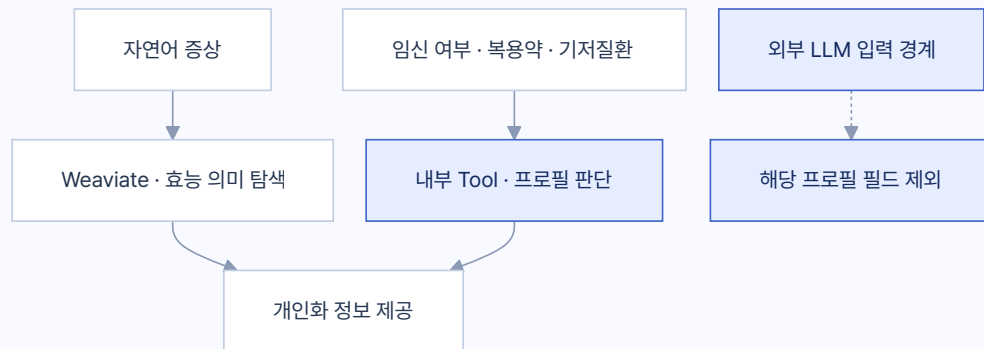
처음에는 건강 프로필을 외부 AI에 보내되 민감한 내용을 걸러내는 필터링을 검토했습니다. 하지만 걸러야 할 표현과 경우가 너무 많아 코드로 모두 막기는 어렵다고 판단했습니다. 그래서 증상 표현을 이해하는 일과 건강 프로필의 조건을 판단하는 일을 분리하고, 프로필을 사용하는 판단은 입력과 규칙을 직접 관리할 수 있는 내부 코드에 두기로 했습니다.

구현

임신 여부·복용약·기저질환 필드를 외부 LLM 입력에서 제외했습니다. 내부 코드의 프로필 판단을 증상 검색 결과와 연결해 개인화 정보를 제공하도록 구현했습니다.

확인한 결과

해당 건강 프로필 필드를 외부 LLM 입력에 포함하지 않으면서, 내부에서는 개인화 판단에 사용할 수 있도록 처리 경로를 분리했습니다.



증상 의미 검색과 건강 프로필의 내부 판단 분리

자연어 증상으로 약품 정보를 탐색하면서도 임신 여부·복용약·기저질환 필드는 외부 LLM 입력에서 제외했습니다. AI에 전달할 정보와 내부 코드가 판단할 정보를 구분한 구현 경험입니다. 민감정보를 다루는 도메인에서 모델에 맡길 판단과 코드가 책임질 판단의 경계를 먼저 정하고 구현했습니다.

결과의 범위 · 트레이드오프

프로필 조건이 바뀌면 내부 코드도 고쳐야 하는 부담이 있습니다. 그래도 민감한 건강 정보가 외부 모델로 나가는 범위를 줄이는 것이 우선이라, 이 판단은 AI에 맡기지 않고 코드에서 처리했습니다. 특정 프로필 항목을 외부 입력에서 뺀 것이며, 모든 경로의 유출 차단을 측정한 결과는 아닙니다.

코드 밖에서도 이어지는 경험

Rhwp · HWPX 저장 오류 수정

Claude Code에 작업 계획과 기여 규칙을 주고 이슈 분석부터 PR까지 자동화해, textFlow 보존 오류 수정(PR #1213) 등 PR 7건이 병합되고 Contributor로 등재됐습니다.

[GitHub 저장소](#) ↗

APPTIVE · 백엔드 멘토링

멘티 경험을 교육 개선으로 연결했습니다. 멘티 12명을 대상으로 6회의 멘토링과 코드 리뷰를 진행했습니다.

백준 945일 연속 문제 해결

하루 한 문제를 목표로 solved.ac 기준 최장 945일 연속 문제를 해결했습니다. 누적 1,659문제, solved.ac Platinum IV.

[풀이 저장소](#) ↗

프로젝트 회고

ARCHITECTURE / 데이터 소유권

MSA 전환과 피드백

기능별 분리에서 데이터 소유권으로 관점을 바꿨습니다. 비동기 처리의 효과와 늘어난 운영 부담을 함께 기록했습니다.

기획 / 사용자 문제 정의

디자인 싱킹 회고

기능부터 고민하던 기획을 돌아보고, 팀원들이 겪은 문서 탐색의 불편함을 모아 함께 해결할 문제를 정의한 과정입니다.

[블로그 글 더보기 ↗](#)

김재형

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com

martinel2.github.io

github.com/Martinel2

LLM 워크스페이스의 AI 기능 전체를 맡아 RAG와 에이전트, 품질 평가 체계를 설계하고, AI 코딩 도구로 구현·검증했습니다.

정확도와 비용, 응답 시간의 트레이드오프를 평가로 확인한 뒤 서비스에 맞는 구현을 선택합니다.